

Document Number: P2009R0
Date: 2019-12-28
Audience: SG16
Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [October 9th, 2019](#)
- [October 23rd, 2019](#)
- [November 20th, 2019](#)
- [December 11th, 2019](#)

October 9th, 2019

Draft agenda:

- P1880R0 - u8string, u16string, and u32string Don't Guarantee UTF Encoding
 - https://github.com/tzlaine/small_wg1_papers/blob/master/P1880_uNstring_shall_be_utf_n_encoded.md
- P1879R0 - The u8 string literal prefix does not do what you think it does
 - https://github.com/tzlaine/small_wg1_papers/blob/master/P1879_please_dont_rewrite_my_string_literals.md
- P1844R0: Enhancement of regex
 - <https://wg21.link/p1844>

Attendees:

- Corentin Jabot
- David Wendt
- Henri Sivonen
- JeanHeyd Meneide
- Peter Bindels
- Peter Brett
- Tom Honermann
- Zach Laine

Meeting summary:

- P1880R0 - u8string, u16string, and u32string Don't Guarantee UTF Encoding
 - https://github.com/tzlaine/small_wg1_papers/blob/master/P1880_uNstring_shall_be_utf_n_encoded.md
 - Zach introduced:
 - The idea is that interfaces taking these string types expect that contents of these strings are well-formed UTF-8, UTF-16, UTF-32 respectively; this requirement needs to be reflected in the standard.
 - We should state a blanket requirement for these expectations.

- The paper proposes a 4th bullet to [\[res.on.arguments\]](#).
- PeterBr asked if the requirement should be for well-formed data.
- Zach replied that it should be. LWG should confirm that.
- Henri asked what happens if an ill-formed code unit sequence is passed. Is it undefined behavior or as-if the Unicode replacement character was present?
- Zach replied that the current wording makes it undefined behavior.
- PeterBr provided an example of why the behavior is undefined. Consider a string that ends with an incomplete code unit sequence; the implementation could run off the end of the buffer.
- Zach responded that, for `std::basic_string` types, the buffer overrun can be avoided, but in that case, the interface specification should state that behavior. The proposed blanket wording is for the weakest interface requirements and can be strengthened by individual interfaces.
- Henri asked if that is useful as it seems like undefined behavior is a huge foot cannon; replacement character semantics would provide a safer interface.
- Zach responded that, if this is a foot gun, then so is `std::vector operator[]`. You must meet preconditions. Implementations can always constrain their handling if they want. The intent here is to enable the fast path.
- PeterBr added that it would add complexity to implement replacement character behavior; interfaces would not be able to use SIMD instructions if ill-formed strings must be handled.
- Zach repeated that the proposal just specifies the default behavior unless otherwise specified for an interface.
- Corentin opined that this seems almost editorial.
- Henri stated that, for `char8_t`, there are values that are never valid in well-formed UTF-8 text and asked what an individual `char8_t` means; it must be restricted to ASCII.
- Tom noted that matches UTF-8 character literals; they can only specify ASCII values.
- Zach read the existing content in [\[res.on.arguments\]](#) in order to demonstrate similarity in existing requirements.
- Henri asked if this represents a requirement that is more difficult to satisfy than the existing requirements. For example, in UTF-16, almost all code bases will allow unpaired surrogates. Does this requirement make the standard library useless for their code bases?
- Zach stated that interfaces can specify their handling of unpaired surrogates.
- Henri asked again if this is a practical requirement.
- Tom responded that this is needed for our mantra of not leaving performance on the floor; we can't both check for ill-formed text and maximize performance.
- Zach added that ICU already does this for performance. Within `Boost.Text`, Zach added interfaces for both unchecked and checked text.
- PeterBr opined that this paper is great and sorely needed.
- Tom and Corentin agreed.
- Henri asked Zach to expound on his statement that ICU already exhibits undefined behavior.
- Zach responded that, in ICU normalization code, assumptions are made when decoding UTF-8. For example, unsafe unpacking of UTF-8 is performed.
- Henri asked if ICU does likewise for UTF-16 for unpaired surrogates.
- Zach responded that he thought so, but is not completely sure.
- Corentin expressed support for an NB comment to include this in C++20.
- Tom opined that it doesn't much matter if this makes C++20 as implementors will already do the right thing.
- Henri asked if this might introduce a backward compatibility issue in C++23 if added after C++20.
- Tom responded that the undefined behavior is effectively already there; this is fixing an underspecification.
- Henri stated it would be a huge task to scrub existing code bases to avoid this undefined behavior.
- Zach predicted that we'll end up with separate interfaces for assuming an encoding vs checking the encoding. This isn't hurting anybody, it is just enabling fast path implementations.

- Henri expressed concern about digging deeper into making default interfaces unsafe; like `std::optional::operator*` is. He would prefer unsafe interfaces be clearly marked as unsafe. This undefined behavior has the potential to introduce security issues.
- Zach responded that most standard interfaces are unsafe in some way, for example every function that accepts arguments of pointer type.
- Henri countered that the undefined behavior can be avoided in this case; just like we could for `std::optional::operator*`.
- Zach suggested that C++ is often used for its performance advantages; we want the default to be fast. But this proposal isn't really about that; it is about documenting our default behavior.
- PeterBr stated that `std::u8string` is `std::basic_string` with `char8_t`. `std::basic_string` provides many interfaces that allow mutating the string in a way that would break otherwise well-formed UTF-8. Rust doesn't do that. We could specify a UTF-8 string type that maintains invariants, but it wouldn't be a `std::basic_string` any more. Thus, it is up to the programmer to not violate UTF-8 requirements.
- Corentin agreed that we don't want to change `std::u8string`; it is just a container of code units. String mutation should be managed via some overlying type like `std::text`. This paper just reflects existing behavior.
- Henri asked if we really want to enable so much performance that we risk our users. In Firefox, lots of string checking is done to avoid security issues even though ill-formed UTF-8 is very rare. The performance isn't bad.
- PeterBr responded that an implementation can choose to define its behavior.
- Henri countered that, if it isn't required everywhere, then it can't be relied on.
- Corentin suggested that, if you want safety, then `std::basic_string` is not the type you're looking for. We're going to need other types on top and, eventually, we'll have more trusted types.
- Zach added that no interfaces are being specified in this paper, so there are no ergonomic concerns. Again, this is just proposing blanket wording that can be strengthened in individual interfaces.
- Tom initiated a discussion about polling during telecons.
 - Tom introduced:
 - He prefers to avoid polling during telecons in favor of polling during face to face meetings. This is due to 1) larger numbers of attendees at face to face meetings, 2) more opportunity for input from those that do not regularly attend telecons, and 3) more opportunity for background thinking after a discussion before having to respond to a poll.
 - He also sees the telecons as useful for priming discussion and identifying non-obvious concerns.
 - Tom asked if anyone wanted to argue for a change in practice.
 - The group expressed general agreement to continue doing what we've been doing.
- P1879R0 - The u8 string literal prefix does not do what you think it does
 - https://github.com/tzlaine/small_wg1_papers/blob/master/P1879_please_dont_rewrite_my_string_literals.md
 - Zach introduced:
 - This started from an experience from a while back that we have previously discussed.
 - Tests involving UTF-8 formatted source files failed when compiled with the Microsoft compiler, but not with other compilers.
 - The source files did not have a UTF-8 BOM and Microsoft's `/source-charset:utf-8` option wasn't being used, so the source files were decoded as Windows-1252.
 - String literals therefore did not contain what was expected because code units were not interpreted as expected.
 - The paper proposes prohibiting use of `u8`, `u`, and `U` literals unless the source file encoding is a Unicode encoding.
 - Corentin suggested relaxing the prohibition to allow use of these literals so long as the source contents of the literal only use characters from the basic source character set. *[Editor's note: presumably this would still allow characters outside the basic source character set if specified with universal-character-name escape sequences.]*
 - Corentin also stated that the current behavior makes sense according to the standard, but most programmers aren't aware of source file encoding vs execution encoding concerns.

- Henri stated that the behavior makes sense if you think of C++ source code as text rather than bytes and agreed that this isn't what programmers expect.
- PeterBr expressed support for the paper because it ensures you get the same abstract characters written in the source file and added that it would be nice if this paper used the same terminology as proposed in Steve's recent terminology paper ([P1859R0](#)). *[Editor's note: this paper will be in the Belfast pre-meeting mailing.]*
- Zach agreed regarding use of terminology.
- Tom expressed concerns regarding breaking backward compatibility, particularly for z/OS where source files are EBCDIC and u8 literals are used to produce ASCII strings.
- Zach asked if it would help to only allow characters from ASCII.
- PeterBr stated that, if the compiler is not explicitly told what the source encoding is, you are in trouble since the compiler can't always detect an encoding expectation mismatch.
- Henri noted that the translation model matches what is done on the web where HTML source is transcoded to some internal (Unicode) encoding. A compiler could preserve meta data about the encoding a literal came from and, if the transcoded code point is above 0x80, issue a diagnostic.
- Zach asked for more information regarding concerns for z/OS and EBCDIC.
- Tom explained the source translation model according to [translation phase 1](#). Source files are first transcoded from an implementation defined encoding to an implementation defined internal encoding. The internal encoding has to be effectively Unicode (or isomorphic to it) due to possible use of universal-character-name sequences in the source code. The internal encoding is then transcoded to the various execution encodings where needed.
- Tom went on to explain that there are multiple EBCDIC code pages and that many of the characters available in them are not defined in ASCII. Restricting UTF literals to just ASCII would prevent use of those characters.
- Tom restated PeterBr's point from earlier. This problem is always due to mojibake; the source file being encoded in something other than what the compiler expects.
- PeterBr agreed that the root cause is the encoding mismatch and opined that this is a problem worth solving. The question is how best to solve it. The first place to look is at the translation from source encoding to internal encoding.
- Henri expressed belief that it makes sense to address the problem where Zach suggests.
- Zach stated that the right place to detect this is during parsing; when parsing a UTF literal, it is critical to know what the source encoding is.
- Tom countered that it is necessary to know the encoding as soon as you hit a code unit that doesn't represent a member of the basic source character set.
- Henri stated that diagnosing any such code unit is a harder sell than just diagnosing one in a UTF literal.
- Tom agreed.
- PeterBr noted that it is implementation defined how (or if) characters outside the basic source character set are represented. The goal of the paper is effectively to tighten that up. That means implementations can have extensions to relax diagnostics.
- Henri responded that such arguments apply to any change to the standard.
- Zach agreed, but noted this is restricted to source files that have UTF literals with transcoded code points outside of ASCII.
- Henri stated that there is more potential for failures for some character sets than others. For example, some character sets don't roundtrip through Unicode. This failure mode already exists, but there is little value in trying to diagnose this outside of UTF literals.
- PeterBr stated that a source file with code units representing characters outside of the basic source character set is ill-formed subject to implementation defined behavior. When a programmer writes a UTF literal, that is a request for a specific encoding, but it is perfectly valid for the source file to be written in Shift-JIS.
- Henri acknowledged that perspective as logically valid, but doesn't address the problems caused by the Microsoft compiler's default behavior not matching user expectations. Programmers are using UTF-8 editors these days.

- PeterBr asserted that is a quality of implementation concern and not an issue with the standard.
- Tom agreed.
- Zach stated that the proposed restrictions can be worked around by using universal-character-name escapes and stated a preference for implementing a solution that results in a diagnosis for the problem he encountered, but that this isn't a critical issue.
- Corentin brought up static reflection and that, at some point, reflection will require defining or reflecting the source file encoding.
- Tom stated that dovetails nicely with Steve's P1859R0 draft that provides a callable for conversion of string literal encoding.
- Corentin noted that Vcpkg compiles all of its packages with the Microsoft compiler's /utf-8 option and that Microsoft may be open to defaulting source encoding to UTF-8 when compiling as C++20.
- Zach added that the Visual Studio editor, by default, adds a UTF-8 BOM to new source files it creates, though it doesn't implicitly add a UTF-8 BOM when existing files are added to a project.
- Corentin observed that, because source encoding is not portable, most programmers just don't use characters outside of ASCII except in comments; which is why such characters are ignored.
- PeterBr suggested that an evening session in Belfast to discuss this or other ideas might be an option and that it would be good to talk directly with implementors.
- Tom confirmed that the next meeting will be on October 23rd and will be the last meeting before Belfast.

October 23rd, 2019

Draft agenda:

- P1844R0: Enhancement of regex
 - <https://wg21.link/P1844R0>
- P1892R0 - Extended locale-specific presentation specifiers for std::format
 - <https://wg21.link/P1892R0>
- P1859R0 - Standard terminology for execution character set encodings
 - <https://wg21.link/P1859R0>

Attendees:

- David Wendt
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Yehezkel Bernat
- Zach Laine

Meeting summary:

- Tom initiated a round of introductions for new attendees.
- P1844R0: Enhancement of regex
 - <https://wg21.link/P1844R0>
 - Tom introduced the paper on behalf of the author:
 - The proposal is an expansion of std::basic_regex specializations.
 - We've discussed issues with std::basic_regex before. The author has put significant effort into this proposal. It includes wording. We owe it to the author to set aside any biases and consider the benefits of this paper.

- An implementation is available though it only implements the proposed `char8_t`, `char16_t`, and `char32_t` specializations, not the existing `char` or `wchar_t` specializations.
- The paper does not propose an alternative to `std::basic_regex`, but rather attempts to address shortcomings of it for UTF encodings via specializations. [*Editor's note: this implies that the proposal doesn't address issues with support of UTF encodings with the `char` and `wchar_t` specializations.*]
- The paper proposes a new regex syntax option, ECMAScript2019, to be used to select a regular expression engine that implements the ECMAScript 2019 specification. This option would be available for use with all `std::basic_regex` specializations.
- The paper proposes a new `dotall` syntax option that allows the `.` character to match any Unicode code point, including new line characters, when using the ECMAScript2019 option.
- The new ECMAScript2019 syntax option would be the only syntax option supported for the `char8_t`, `char16_t`, and `char32_t` specializations.
- The ECMAScript2019 regular expression engine would **NOT** exactly match the ECMAScript 2019 specification:
 - The `\xHH` expression is redefined to match code points rather than code units. However,
 - The author would be fine with removing support for the `\xHH` expression since support for code points is provided by the `\uHHHH` and `\u{H...}` expressions.
- The proposal removes locale dependency for the `char8_t`, `char16_t`, and `char32_t` specializations and therefore does not propose any new specializations of `std::regex_traits`.
- The paper proposes new overloads of `std::regex_match` and `std::regex_search` to allow specifying look behind limits on ranges.
- The proposed changes to `std::regex_iterator` are ABI breaking.
- PeterBr observed that the proposal doesn't deal with language specific aspects like case folding.
- PeterBr stated he liked the motivation for this paper and the notion that `std::regex` can be made to work.
- Zach asked about support for collation and whether anyone was familiar with the existing `collate` syntax option.
- PeterBr responded that the paper states that the `collate` option is ignored for these specializations.
- Zach stated that the default collation is not useful and that tailoring is required.
- Tom summarized, so the paper needs to address collation.
- Zach refuted that need since it could profoundly impact performance.
- PeterBr suggested that, perhaps, regex for Unicode should operate on `std::text`.
- Tom expanded that suggestion to any sequence of code points and observed that the proposal kind of does that already via the changes to `regex_iterator`.
- Zach agreed it would be useful to use as an adapter for code points.
- Tom asked if a new regex feature for non-compile-time regex support would be preferred over specializing `std::basic_regex` as proposed.
- Zach responded that he doesn't think `std::regex` is DOA, but if we're going to support Unicode regex with dynamic patterns, then, we should pursue some of the design of CTRE.
- Zach added that solving the problem is important and that he wants to see Unicode regex support but would prefer to take a wait-and-see approach on this paper while watching how CTRE and `std::format` evolve.
- PeterBr acknowledged the benefits of CTRE, but stated that we do need a solution for dynamic regex.
- Zach reported that he believes that Hana is planning to make CTRE capable of supporting dynamic pattern strings and if that were to happen, then we wouldn't need `std::regex` any longer.
- Mark lamented the lack of a proposal like this one when C++11 was being designed since the approach looks good relative to other papers from the past.
- Mark added that it is an embarrassment that we don't have a solution for this today, but that he feels kind of neutral on it as well due to concerns about allocating time for this relative to other things we could do.
- Mark asked what implementors would think and if they get requests for Unicode `std::regex` support.
- Mark asserted that the implicit use of the ECMAScript2019 engine when a different syntax option is specified has to be changed.

- Zach reiterated that this proposal is definitely an ABI break, that an ABI break is a serious problem, and that the need for such a break suggests we need a different family of types.
- Mark added that the paper should make it clear that it does break ABI, not that it might.
- Tom asked if this proposal solves the `std::basic_regex` issues with support for variable length encodings.
- Zach responded that `std::regex` doesn't handle incomplete or ill-formed code unit sequences and suggested that perhaps those should match against `\uFFFFD`.
- Zach reported that `std::regex` can also match code unit ranges that stride code unit sequences since `std::regex` effectively matches bytes.
- Tom asked what guidance we should offer to LEWG.
- Zach suggested:
 - We should solve this problem.
 - This approach is premature given other things in flight now, but if this had been proposed three years ago he might have felt differently about it.
- PeterBr suggested it should be prioritized behind CTRE.
- Tom asked whether support for tailoring is important.
- Zach suggested placing tailoring at the lowest priority and mentioned that he doesn't think ICU supports it as people don't often want to do collation aware searching.
- Tom reiterated that we should offer guidance that it be ill-formed to specify a syntax option other than ECMAScript2019 for the proposed specializations.
- P1892R0 - Extended locale-specific presentation specifiers for `std::format`
 - PeterBr introduced the paper:
 - Looking through the `std::format` specification he found that there are useful floating point formats that can not be produced in locale specific formats.
 - Locale specific formats are important in scientific fields.
 - The 'n' specifier has a different meaning for integers than it does for floating point.
 - An NB comment was filed to make the 'n' specifier indicate a locale specific format rather than a type modifier.
 - The proposed change should not affect existing well-formed `std::format` calls except for `bool` which would now be formatted as locale variants of "true" or "false" instead of 1 or 0.
 - This would make `std::format` unambiguously the best choice for localized formatting since locales can be easily specified and `std::format` already solves short falls of `iostreams` and `printf` such as ordering.
 - Without this change, there is still a need to use `printf` for locale sensitive formatting.
 - Mark noted that this change will break existing users of `{fmt}`.
 - PeterBr responded that it will for existing uses of `bool` but that he isn't concerned about existing users of `{fmt}`.
 - Tom observed that use of 'l' as the specifier as suggested in the paper avoids the break and aligns with Victor's [P1868R0](#) paper to enable locale specific handling of character encodings.
 - Mark stated that the core issue is that there remains some uses of `printf` that can't be directly replicated with `std::format` and asked how a programmer would print, for example, the locale specific decimal character but without the locale specific thousands separator.
 - PeterBr responded that the programmer can create a custom locale.
 - Zach stated that we can't defer this until C++23 because changing the meaning of 'n' would break compatibility and asked why we can't just introduce an 'l' specifier in C++23?
 - PeterBr responded that doing so makes things more complicated and asked whether we would deprecate 'n' if 'l' were to be adopted. We can postpone addressing this, but we get a cleaner solution in the long term by addressing it now.
 - Zach agreed with the motivation being to avoid a wart that we'll need to teach but that some opposition will be raised due to perceived risk at this late stage.
 - Zach stated that he likes the change, but that it needs good motivation.
 - PeterBr suggested that 'n' could be removed now and then restored with desired changes in C++23.

- Zach suggested that if Victor supports the paper, it will probably pass, but if he disagrees with it, then it is probably DOA.
- Mark stated that the choices need to be clearly presented for LEWG.
- Zach observed that there are a few options and suggested presenting a cost/benefit of each so that LEWG is given clear choices.
- Mark suggested socializing the issue on the LEWG mailing list now to flush out any objections.
- PeterBr stated that any help improving the paper would be appreciated.
- Mark suggested presenting either slides or a different paper that presents the options and analysis.
- PeterBr stated he would create a doc that could be collaboratively edited.
- P1859R0 - Standard terminology for execution character set encodings
 - Steve introduced the paper:
 - The goal is to not affect implementations, but rather to fix wording so that we can use modern terminology and understand each other better.
 - We often use terms like "execution encoding" that are not defined in the standard and are opportunities for confusion.
 - We need to admit that `wchar_t` is not, in practice, able to hold all code points of the wide execution character set.
 - Zach asked what "literal encoding" is for.
 - Steve responded that it reflects the encoding for non-UTF literals.
 - Zach asked what difference is intended by "character set" and "character repertoire".
 - Steve responded that the goal is to tighten up the meanings of existing terminology so as to avoid massive changes to the standard.
 - Mark observed that there seems to be a missing word in the definition of "Basic execution character set"; that there seems to be a missing "that".
 - PeterBr stated that this should be high priority in C++23 so we can get everyone on board with terminology.
 - Steve agreed and asserted we'll need to socialize these new terms.
 - Tom asked if there are any terms being dropped; it looks like the paper adds "literal encoding" and "dynamic encoding".
 - Steve responded that none are dropped and stated there will be an additional associated encoding added for character types as well.
 - Mark noticed that the paper discusses `literal_encoding` and `wide_literal_encoding` but doesn't define a term for "Wide literal encoding".
 - Tom asked if "source encoding" should be added.
 - Tom asked if we should add a statement that the dynamic encoding must be able to represent all of the characters of the execution character set.
 - Steve responded that we could add that.
 - PeterBr observed a potential problem with doing so on Windows where the dynamic encoding might be UCS-2, but the execution character set is UTF-16.
 - Tom suggested refining the requirement such that characters used in literals must have a representation in the dynamic encoding.
 - Mark suggested it would be helpful to have a cheat sheet with mathematical notation of which terms denote a subset of other terms.
 - Steve agreed.
 - Tom suggested that we also need "wide dynamic encoding".
 - Zach asked about the difference between the "encoding" and "character set" terms.
 - Steve responded that the former states how characters are represented while the latter states what characters must be representable.
 - Zach stated it would be useful to have text explaining the difference.
 - Tom asked how ODR violations would be avoided for `literal_encoding` since literal encoding can vary by TU.

- Steve responded that the same technique used for `std::source_location` can be used; a value is provided.
- Tom confirmed that the next meeting will be November 20th.

November 20th, 2019

Draft agenda:

- Belfast follow up and review.
- Volunteers to draft a library design guidelines paper.

Attendees:

- JeanHeyd Meneide
- Mark Zeren
- Steve Downey
- Tom Honermann
- Yehezkel Bernat
- Zach Laine

Meeting summary:

- [P1868 - !\[\]\(d691176a0c8c25c55a5ec880b4a44829_img.jpg\) width: clarifying units of width and precision in std::format:](#)
 - Tom introduced the topic:
 - Concerns were raised in Belfast with regard to the stability of the proposed code point ranges to be used for display width estimation. The currently proposed ranges map all extended grapheme clusters (EGCs) to a display width of one or two despite there being a number of known cases of EGCs that consume no display width (e.g., U+200B {ZERO WIDTH SPACE}) or more than two display width units (e.g., U+FDFD {ARABIC LIGATURE BISMALLAH AR-RAHMAN AR-RAHEEM}).
 - Additionally, the EGC breaking algorithm is dependent on Unicode version and the proposed wording does not specify which version of Unicode to implement. Concerns were raised regarding having a floating reference to the Unicode standard and the potential for differences in behavior across implementations if the Unicode version is implementation defined and subject to change across compiler versions.
 - How should we address these concerns?
 - Zach commented that the wording review went through LWG ok and that he had posted a message to the LWG mailing list responding to one concern that was raised.
 - Zach reported that Jonathan Wakely stated that floating references to other standards are not permitted but that implementors can, as QoI, offer support for other versions.
 - Tom expressed surprise regarding that restriction given that we have a floating reference to ISO 10646 in the working paper today.
 - Zach responded that LWG stated a requirement for a normative reference and is therefore planning to add a normative reference to Unicode 12 with the intent that we update the normative reference with each standard release.
 - Tom asked that, if we reference a particular version, can implementations use a later version and remain conforming.
 - Zach responded that doing so seems to be acceptable to implementors.
 - Steve remarked that CWG expressed a preference for a floating reference.
 - JeanHeyd confirmed and added that is how the working paper ended up with the floating reference to ISO 10646.

- Zach said he will follow up about this discrepancy.
- Mark asked if we have a preference for floating vs fixed.
- Zach responded that implementations will do what they need to do for their users.
- Tom turned the discussion back to concerns raised by Billy regarding changes to the width estimate algorithm being a breaking change; e.g., changing the width estimate for a given EGC. This is a related but distinct concern from the EGC algorithm changing due to a change in Unicode version.
- Zach stated that U+FDFF is an example of something we need to fix that can also be a breaking change.
- Steve repeated that the concern is basically any change in behavior potentially resulting in a surprising or undesirable change.
- Mark asserted that we're going to continue having difficulties with dependencies on Unicode data and that the situation is analagous with respect to the timezone database. Implementors can enable stable behavior by allowing choice of Unicode version.
- Steve noted that the rate of change of the Unicode standard has skewed towards stability.
- Mark opined that we should not solve this problem in the standard.
- Tom agreed and added that we can specify a minimum version, but leave the actual version implementation defined.
- Mark asked which version of the Unicode standard the proposed code point ranges were pulled from.
- Tom responded that the Unicode standard doesn't contain character display width data and that these were extracted from an implementation of `wcswidth()`.
- Steve stated that he maintained a list of double wide characters for years and that it was not a significant burden.
- Tom stated that his desire for a floating reference to the Unicode standard with an implementation defined choice of version is intended to allow implementors to keep up with new Unicode versions. Unicode releases happen every year while C++ standards are only released every three years. Implementors probably can't lag Unicode by three years.
- Zach acknowledged the goal and stated that will result in some implementation divergence as some implementors will keep up and some won't, but that the differences are likely to be minor.
- Tom asked if ISO 10646 annex U constitutes a reference to [UAX#31](#).
- Steve suggested this is probably a bureaucratic issue and added that having a normative reference is helpful.
- Zach responded that it could be harmful if we get conflicting floating and non-floating references for ISO 10646 vs Unicode, but this should fall to LWG and CWG to decide.
- Tom asked how we should go about fixing the currently proposed width estimates since the proposed ranges are clearly missing support for cases of zero width or width greater than two.
- Zach opined that he wasn't sure there is a problem to be fixed since what is specified matches existing practice.
- Tom asked if we know where this implementation of `wcswidth()` came from and how widely deployed it is.
- Zach suggested asking Victor.
- [Editor's note: According to [P1868R0](#), the implementation of `wcswidth()` is the one at <https://www.cl.cam.ac.uk/~mgk25/ucs/wcwidth.c>.]
- Tom asked for opinions regarding writing a short paper that explains the Unicode stability guarantees and argues for floating references and implementations.
- Zach suggested waiting for a more motivating reason to do so.
- [P1949 - C++ Identifier Syntax using Unicode Standard Annex 31](#):
 - Tom introduced the topic:
 - EWG rejected the SG16 guidance offered in response to NB comment [NL029](#) to deprecate identifiers that do not conform to [UAX#31](#) with noted exceptions for the `_` character.
 - A suggestion was made that a CWG issue be filed to consider the lack of updates to the allowed identifiers since C++11 as a defect.
 - Tom agreed to file a core issue and started to do some research.

- According to [N3146](#), the original identifier allowances appear to have been aggregated from various sources including [UAX#31](#) and [XML 2008](#), and following guidance in annex A of a draft of [ISO/IEC TR 10176:2003](#).
 - Thank you to Corentin for quickly providing a way to query the code point ranges that have the `XID_Start` or `XID_Continue` property set. <https://godbolt.org/z/h7ThEh>. These ranges differ substantially from what is in the current standard.
 - What should the proposed resolution for the core issue be?
- Steve stated that [UAX#31](#) permits extensions, and what was adopted for C++11 effectively whitelisted a large set of code points.
- Zach asked what EWG's concern was.
- Steve replied that they were nervous about such a late change and want more time to think it through.
- Zach opined that this seems like something better addressed in C++23.
- Steve noted that what is done can be back ported to prior standards though, that Clang and gcc support Unicode encoded source code [*Editor's note: so does MSVC*], and that the longer we wait to address this, the more code we potentially break.
- Tom stated that, from the DR perspective, we could either figure out what we want for C++23 and recommend that as the proposed resolution, or we can do a more targeted fix for C++20 for specific problematic cases knowing that we'll likely do differently for C++23.
- Steve stated that the only difference C++ needs from [UAX#31](#) is support for `_`, and such an extension is conforming. It would also be ok to restrict identifiers to a common script to avoid homoglyph attacks.
- Steve added that there is also the issue of normalization forms and that gcc will currently warn if identifiers are not in NFC form.
- Mark asked if we should make it ill-formed for identifiers to not be in NFC form.
- Steve responded that doing so could break existing code.
- Tom suggested normalizing when comparing identifiers is another approach.
- Steve noted that doing so requires the Unicode normalization algorithms.
- JeanHeyd mentioned that we'll also have the problem of reflecting identifiers in the future and that normalization will be relevant there. Corentin brought this up in SG7. Requiring NFC would be helpful there.
- Mark expressed support for the idea of requiring NFC.
- Steve suggested that there is always the universal-character-name escape hatch.
- Mark opined that EWG probably won't like requiring conversion to NFC in name lookup.
- Tom responded that gcc is at least detecting non-normalized identifiers today, that doing so must require some level of Unicode database support, and that performance costs are presumably reasonable.
- Steve stated that gcc looks for some range of combining code points and may not be 100% accurate.
- Mark asked if non-NFC normalization can be detected without having to fully normalize?
- Zach responded that he didn't think so.
- Mark asked if normalization was brought up in EWG.
- Steve responded that it wasn't, that we didn't get that far in the discussion.
- Tom suggested that we have a good amount to think about here and that he is looking forward to the next revision of Steve's paper.
- Steve took the bait and agreed that the paper will have to provide good arguments for why this is important.
- Zach suggested that this should be easy for implementors if they don't have to deal with normalization and that we should just require NFC for performance reasons.
- Mark asked if we could make use of non-NFC ill-formed NDR so that implementations are not required to diagnose violations.
- [P1097 - Named character escapes](#):
 - Tom introduced the topic:
 - EWG narrowly rejected the paper, but expressed good support for the direction.
 - Most concerns had to do with implementation impact and, in particular, the potential increase in compiler binaries. Some distributed build systems distribute compilers as part of the build process

and the additional latency imposed by increasing the size of compiler binaries adds cost. Numbers haven't been obtained, but guesses were around 2MB, but could probably be reduced to under 600K.

- One prominent EWG member was strongly opposed to the design because he would prefer a solution that avoids baking Unicode into the core language. Something like a string interpolation solution that could call out to constexpr library functions to do character name lookup.
- Martinho was working on an implementation in Clang at Kona, but Tom doesn't know the state of it or where to find it. Tom reached out to Martinho via email, but didn't hear back.
- Anyone have time and interest to experiment and produce some estimates to address the implementation impact concerns?
- Steve stated that he could probably do some work on it and that the name DB should compress really well with use of a trie.
- JeanHeyd suggested that the [UAX44-LM2](#) compression scheme could help to reduce size.
- Tom expressed uncertainty that it would help much over a trie, but we could experiment and put the results in a paper.
- Zach suggested splitting names that contain "with" in them since the suffixes that tend to follow "with" are highly repeated.
- Tom noted that the algorithmically generated names could be specially handled as well.
- Steve added that a tokenization approach could help too.
- Tom asked if anyone might know of a link to Martinho's implementation.
- Zach replied that a link was provided at some point, possibly in Slack.
- *[Editor's note: Tom searched Slack, but failed to find a reference.]*
- [P1880 - uNstring Arguments Shall Be UTF-N Encoded](#):
 - Tom introduced the topic:
 - LEWG rejected the SG16 guidance offered in response to NB comment [FR164](#) to adopt P1880 for C++20.
 - What should we do next?
 - Zach expressed frustration that he was available when the NB comment and paper were discussed in LEWG, but that no one notified him that the discussion was happening.
 - Zach stated that, after the SG16 meeting, he went through all references to std::basic_string and added missing references to PMR strings and std::basic_string_view. This research also identified a number of references that are deserving of more scrutiny.
 - Zach opined that this isn't very important for C++20 and that he will work on a revision for C++23, though not for the Prague meeting.
 - Zach stated he was surprised at how many references to these types he found in function templates.
- Tom asked for volunteers to draft a library design guidelines paper.
 - Tom introduced the topic:
 - During the [SG16 meeting on July 31st](#), we discussed guidelines for when to add function overloads for each of char, wchar_t, char8_t, char16_t, and char32_t and he would like to have a library guideline paper that records our guidance.
 - Would anyone be interested and willing to work on this?
 - Zach expressed interest in doing so.
- Mark brought up a wording update email Zach sent to LWG with regard to [P1868](#):
 - Mark noted that the wording introduces a new term of art: "estimated display width units".
 - Zach responded that the new term was intentional; we're leaving the width estimation effectively unspecified for non-Unicode encodings. Implementors expressed a preference for not having to document their choices and we didn't want to force embedded compilers to have to be Unicode aware. So, we needed a non-Unicode term.
 - Tom noted that the wording appears to require embedded compilers to use the proposed Unicode algorithm if their execution character set is Unicode.
 - Zach acknowledged that would be the case.
 - Mark suggested that is probably what we want if they are actually doing Unicode.

- Tom agreed and suggested such implementors could otherwise state that their execution character set is ASCII.
- Tom communicated that the next meeting will be on December 11th.

December 11th, 2019

Draft agenda:

- Vocabulary type(s) for extended grapheme clusters?
 - Per Michael McLaughlin's questions posted to the (old) mailing list on 11/01.
- P1097: Named character escapes
 - Review research on minimizing the name lookup DB and code size.

Attendees:

- Corentin Jabot
- David Wendt
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann

Meeting summary:

- [P1097: Named character escapes](#):
 - Tom introduced the topic:
 - Since our last meeting, Corentin did some outstanding investigative and evaluation work and blogged about his results:
 - https://cor3ntin.github.io/posts/cp_to_name
 - Corentin's implementation of his size reduction techniques is available at:
 - https://github.com/cor3ntin/ext-unicode-db/tree/name_to_cp
 - The goal for today is to review his results and determine next steps.
 - Corentin opined that the data is still kind of large at approximately 260K.
 - Zach noted that Corentin did a good job of estimating a theoretical lower bound for reducing the data at around 180K, so achieving a result of 260K is great.
 - Steve commented that the code shows the challenges C++ has with variable length data. The natural representation would use variants, but that can't be represented as well.
 - Corentin agreed noting that good performance demands working at the byte level.
 - Zach expressed a similar experience working on [Boost.text](#); flat arrays of bytes had to be used to achieve scaling goals.
 - Tom stated that we need to draft a revision of this paper and that he is happy to do so, but would welcome any other volunteers.
 - Corentin asked if we know how to get in touch with Martinho.
 - Tom responded that he tried, but did not get a response.
 - Tom noted that, if we can't get in touch with Martinho, then we'll need to submit a new paper rather than a new revision.
 - Corentin asked if a new paper was really necessary.
 - Steve responded that, as a matter of procedure, we need a new paper to get it on the schedule.
 - PeterBi added that we need a place to record the new information.
 - Tom stated he would attempt to contact Martinho again.

- *[Editor's note: Tom did reach out again via email, but again did not get a response.]*
- Tom asked Corentin if he wanted to take this and run with it given the considerable investment he has already made.
- Corentin responded that he is unfortunately time constrained.
- Corentin mentioned that the new paper should state the need for matching name aliases and case insensitivity.
- Tom agreed and noted that we have polls on those topics from presentation to EWGI in San Diego that record a trail of intent for those cases.
- Zach asked Corentin if dashes are handled properly in his experiment.
- Corentin replied affirmatively that spaces, dashes, and underscores can be omitted or swapped as recommended by Unicode in [UAX44](#).
- Corentin added that the current 260K size includes support for name aliases.
- Steve observed motivation for allowing spaces, dashes, and underscores to be swappable; that behavior falls out of a good implementation.
- Corentin stated that, should a desire arise to be able to map code points to names, then a different implementation would provide a more optimized data set that handles mapping both directions.
- Tom asked Corentin for an estimated size for a perfect hash approach.
- Corentin responded with 300K to 400K.
- Corentin pointed out a potential challenge; that it may be desirable to support code point to name mapping in the standard library, but probably not in the compiler. This implies a potential need for the Unicode character name data to be available to both.
- Steve stated that it seems unfortunate to not expose the compiler data to the library.
- Corentin suggested the data would probably need to be present in both the compiler and the library.
- Tom provided a possible way to avoid that; by making it available in the library, but accessible from the core language. At least one EWG member strongly advocated for such an approach; a string interpolation like facility.
- Vocabulary types for extended grapheme clusters:
 - Tom introduced the topic:
 - Michael McLaughlin had posted some questions to the (old) mailing list on 2019-11-01:
 - <http://www.open-std.org/pipermail/unicode/2019-November/000868.html>
 - These questions are related to representation of extended grapheme clusters (EGCs), specifically, how a collection or sequence of them might be stored.
 - Should the standard library provide vocabulary types for EGCs?
 - Zach explained the choices he made for [Boost.text](#). There are two vocabulary types; [grapheme](#) provides value semantics and stores a small vector optimized sequence of code units with a maximum size limited according to the [Unicode stream-safe text format described in UAX #15](#), and [grapheme_ref](#) provides read-only reference/view semantics over a code point range denoted by an iterator pair.
 - Zach added that he is unsure if anyone is using the value type.
 - Corentin acknowledged the uncertainty regarding use cases for a value type.
 - Corentin asked why the reference/view version is not an alias of a span.
 - Zach responded that he wanted to support subranges and non-contiguous storage. The implementation uses the `view_interface` CRTP base from C++20 ranges.
 - Steve asked who the anticipated consumers are for use of EGCs.
 - PeterBr expressed similar curiosity and provided some background experience; he previously worked on a product that was text based and everything was done on graphemes. Support was available for individual grapheme replacement, but a value type was never needed because reference/view semantics were always desired. All text processing was performed in terms of ranges of graphemes.
 - Zach offered a couple of examples. Text rendering depends on knowledge of EGC boundaries. Additionally, an EGC reference is the value type of an (EGC-based) iterator on a text range.
 - Zach observed that breaking algorithms don't always break on EGC boundaries, though split EGCs still remain EGCs on either side of the boundary.

- Steve stated that having a named type is very useful. An EGC view is essentially a subrange, but naming it is useful.
- PeterBr clarified that an EGC is effectively a range of code points.
- Tom asked if there is a good distinction between an EGC type that represents a range of code units or code points that constitute exactly one grapheme vs a type that represents a range of EGCs in terms of a range of code units or code points.
- Zach replied yes, [Boost.text](#) has a type that represents the latter case as well; [grapheme_view](#) is a view that provides an EGC iterator. So, yes, there are three potentially useful types: an owning EGC, a reference EGC, and an EGC view.
- Steve asked how breaking algorithms that split EGCs interact with these types.
- Zach replied that all Unicode algorithms are specified in terms of code points, not EGCs. So, a split EGC just becomes two EGCs. The sentence breaking algorithm may cause this to happen.
- Tom recalled prior conversations where we discovered that the EGC sum of the parts of a text may be greater than the EGC sum of the whole text.
- Steve asked for confirmation that you can still view the split code point ranges as EGCs.
- Zach confirmed, yes.
- Corentin asked if all of these types aren't effectively subranges.
- Steve replied yes, but different types is useful to avoid subranges of subranges.
- Corentin countered that, if you have a `text_view` and you split it, you get a `text_view`.
- Zach stated that the idea that the Unicode algorithms produce sequences of code points but programmers want EGCs is a key idea.
- PeterBr observed that rendering text requires more than just EGCs.
- Steve returned conversation to the motivation for EGC types and mentioned the DB field example; there is a known limit of how many bytes can be stored, EGCs indicate where text should be truncated to.
- Tom asked if there is a need to distinguish between an EGC view and a subrange of EGC view other than an EGC reference; as Corentin mentioned, a subrange of a `text_view` is a `text_view`, so is a subrange of an EGC view an EGC view?
- Zach stated he didn't see a need for such a distinction. Most interfaces should operate on EGC views, but for Unicode algorithms, it is necessary to drop down a level to a code point view.
- Steve summarized; an EGC reference is a view over code points with a contract that its range represents exactly one EGC.
- PeterBr imagined a scenario in which a range of code points is sliced to produce multiple EGCs, but when recombined with additional text, might yield different EGCs.
- *[Editor's note: Some discussion was missed here.]*
- Tom stated a need for consistent terminology. Tom originally proposed `text_view` as a sequence of code points, but we now think it should be EGC based.
- PeterBr expressed concern; most people think they want code points. LEWG might object to an EGC based design.
- Zach stated that a concern we have is that we're the Unicode experts and everyone with strong opinions is pretty much on this call; we need to be aware of echo chamber issues.
- Tom added that echo chamber issues are the thing that keeps me up at night; how do we ensure we deliver what is truly useful?
- Steve added that he frequently is asked why some simple thing isn't implemented. The answer is, because it isn't actually simple.
- Corentin stated that he gets quite concerned whenever we discuss going in a direction that doesn't align with Unicode recommendations; the UTC (Unicode Technical Committee) doesn't get things wrong very often.
- Steve noted that, fortunately, we're kind of late to the game, we can learn from the experience of other languages, and we don't have to discover all the problems ourselves.
- Tom returned discussion to the subrange of subrange concern; there may be a need to put subranges back together.

- Corentin replied that there is an ongoing effort to support that, but it is complicated. JeanHeyd is working on [P1664](#) and it should be discussed more in Prague.
- Steve described one of the challenges; for efficiency, when we have an EGC view and want to get down to the code unit range for efficient IO, reassembly can get difficult.
- Zach replied that, if you have an EGC view over a code point view over a sequence of code units, that is easy.
- Tom countered that doing so requires that you know that the underlying storage is contiguous if you want to operate on it at the code unit level.
- Steve added that there can't be a missing range in the middle.
- Corentin expressed a belief that this will be solved; maybe not for C++20, but for C++23.
- Tom stated that our normal meeting cadence would have us meeting again on December 25th 🎅, but expected meeting that day would be unpopular, so we'll plan to meet next on January 8th.